The alternative is to provide iteration functions such as `for_each` as part of the container itself. This puts the container squarely in charge of the iteration and locking, but it does fall foul of the deadlock avoidance guidelines from chapter 3. In order for `for_each` to do anything useful, it must call user-supplied code while holding the internal lock. Not only that, but it must also pass a reference to each item to this user-supplied code in order for the user-supplied code to work on this item. You could avoid this by passing a copy of each item to the user-supplied code, but that would be expensive if the data items were large.

So, for now you'll leave it up to the user to ensure that they don't cause deadlock by acquiring locks in the user-supplied operations and don't cause data races by storing the references for access outside the locks. In the case of the list being used by the lookup table, this is perfectly safe, because you know you're not going to do anything naughty.

That leaves you with the question of which operations to supply for your list. If you cast your eyes back on listings 6.11 and 6.12, you can see the sorts of operations you require:

- Add an item to the list.
- Remove an item from the list if it meets a certain condition.
- Find an item in the list that meets a certain condition.
- Update an item that meets a certain condition.
- Copy each item in the list to another container.

For this to be a good general-purpose list container, it would be helpful to add further operations such as a positional insert, but this is unnecessary for your lookup table, so I'll leave it as an exercise for the reader.

The basic idea with fine-grained locking for a linked list is to have one mutex per node. If the list gets big, that's a lot of mutexes! The benefit here is that operations on separate parts of the list are truly concurrent: each operation holds only the locks on the nodes it's actually interested in and unlocks each node as it moves on to the next. The next listing shows an implementation of just such a list.